

iOS/iPadOS App FlipByBlink

version 3.1.1

2022-12-28

目次 Index

プライバシーポリシー Privacy Policy 4

サポートデバイス Support device 5

このアプリについて About this App 6

- 概略 Outline 6
- AppStore URL 6
- サポートOS Suport OS 6
- 主な機能 Main functions 7
- キーボード対応 Keyboard support 9
- リリース元/開発者 Publisher/Developer 9
- サポート言語 Support language 9
- アップデートの予定 Updates plan 9

背景 Background 10

- おおまかな経緯 Whole story 10
- both + slow + wink + blink → “boslowink” 11

透明性レポート Transparency report 13

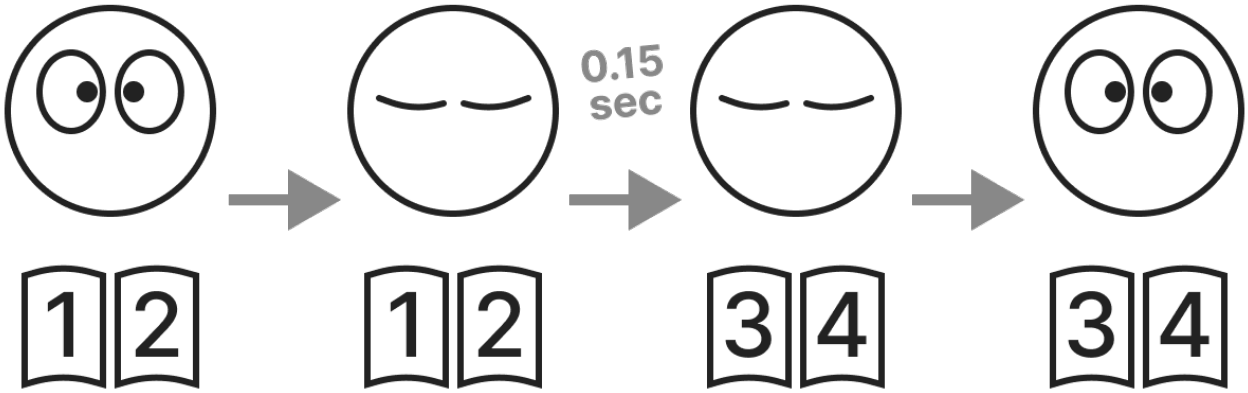
- AppStoreリリースに関して AppStore release 13
- バージョン履歴 Version history 14
- 卒業論文(2019) Graduation thesis(2019) 19
- グラフィックデザイン資料 Graphic design resource 20

よくありそうな質問 FAQ 21

謝辞 Acknowledgments 24

- ZIP Foundation 24

ソースコード Sourcecode 25


0.15
sec
1 2
1 2
3 4
3 4

プライバシーポリシー

Privacy Policy

日本語

本アプリケーション FlipByBlink では下記のAPIを用いて顔認識を実行します。

・Apple Inc. “ARKit TrueDepth API”

これらの情報はページめくり機能のみを目的として端末上だけで活用します。取得した情報を永続的に保存することはありません。また、それらの情報が本アプリケーション制作者や第三者に渡ることはありません。

参照: https://developer.apple.com/documentation/arkit/arfacetrackingconfiguration

English

This application will use the API to perform face tracking.

・Apple Inc. “ARKit TrueDepth API”

These data will be used only on the device for the purpose of page flipping feature only. The application will not store any of that data. It will not be passed on to the application's creator or any third party.

ref. https://developer.apple.com/documentation/arkit/arfacetrackingconfiguration

サポートデバイス

Support device

瞬目を検出するためにApple製フレームワークであるARKitのFaceTracking APIを使用しています。そのため TrueDepthカメラ、もしくはA12 Bionic以降のチップが必要です。

To detect your blinking eyes, this app uses FaceTracking API of Apple's framework ARKit. Therefore, it requires TrueDepth camera OR A12 Bionic or later chip.

【参照 ref.】https://developer.apple.com/documentation/arkit/arfacetrackingconfiguration

サポート対象外のデバイス

UNSUPPORTED DEVICE

- **iPhone** 6s, 6s plus, 7, 7 plus, 8, 8 plus, SE 1st
- **iPad** 5, 6, 7, Air 2, mini 4, Pro 9.7, Pro 10.5, Pro 12.9 1st, Pro 12.9 2nd
- **iPod touch** 7

このアプリについて

About this App

概略 Outline

- シンプルで一般的な電子書籍リーダー(固定レイアウト向け)
- 特殊な機能は1つだけ。目を閉じる時間を意識的にわずかに長くさせた瞬きでページめくりが出来ます
- PDFファイルやZIPファイルに対応
- カメラであなたの瞬きを認識
- 無料、無広告、無トラッキング
- *Simple and normal ebook reader (for fixed-layout)*
- *Only a special feature. Flip a page with slightly longish voluntary blink*
- *Support PDF / ZIP file format*
- *Recognize your blinking eyes using a camera*
- *Free, non ad, non tracking*

AppStore URL

https://apps.apple.com/jp/app/id1444571751

サポートOS Suport OS

iOS / iPadOS ver 15.0 ~

主な機能 Main functions

- ファイルをインポート
- ファイルを読み始める
- アプリドキュメント本を開く
- アプリドキュメント動画を開く
- 「設定」アプリへ移動
- 本アプリのAppStoreページへ移動
- *Import PDF*
- *Start reading book*
- *Open “App document book”*
- *Open “App document video”*
- *Jump “Setting” app*
- *Jump AppStore page*



- 次のページへ移動
  - 目を閉じる時間を意識的にわずかに長くさせた瞬目
  - 画面の右側をタップ
  - 右から左へスワイプ
- 前のページへ移動
  - 画面の左側をタップ
  - 左から右へスワイプ
- ピンチイン・ピンチアウトで高速ページ移動
- 上から下へスワイプで本を閉じる
- 下から上へスワイプで任意のページ番号へ移動
- *Go to next page*
  - *slightly longish voluntary blink*
  - *tap right side*
  - *swipe right to left*
- *Go to previous page*
  - *tap left side*
  - *swipe left to right*
- *Pinch in/out to turn quickly*
- *Swipe down to close book*
- *Swipe up to jump appointed page*



想定対象ユーザー Target users

固定レイアウト方式の電子書籍を読むすべての人

All people who read fixed-layout ebooks

ユースケース Use cases



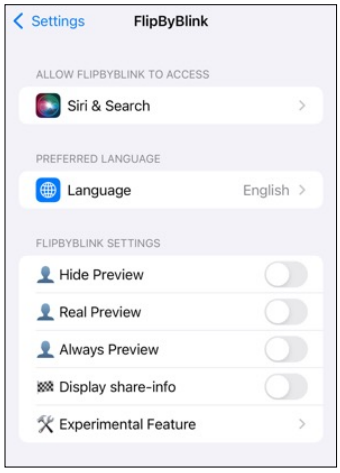
- 一般的な読書
- 楽譜を見ながら楽器を演奏
- レシピ本を確認しながら料理
- 電車やバスでの読書
- 身体障害を抱える方の読書

など

- *General reading*
- *Playing an instrument while looking at music score*
- *Cooking while checking a recipe book*
- *Reading on the train or bus*
- *Reading by people with physical disabilities*

etc.

オプトイン機能 Setting options



- プレビュー機能のAR映像をカメラ映像に変更
- プレビュー機能の表示が消えないように固定
- 画面収録用にアプリ情報を常時表示
- [実験的な機能] ページめくりに必要な時間の調整機能
- [実験的な機能] 片目を長めに閉じて1ページ戻る
- *Change AR image of preview to camera image*
- *Always display the preview*
- *Always show app information for screen recording*
- *[Experimental feature] Setting threshold*
- *[Experimental feature] Return 1 page with one eye closed*

キーボード対応 Keyboard support

+ 1 page :　→ , d , spacebar , pagedown

+ 5 page :　↓ , s

- 1 page :　← , a , shift*spacebar , pageup

- 5 page :　↑ , w

リリース元/開発者 Publisher/Developer

山下 亮 (1人のみ)

Yamashita Ryo (only)

サポート言語 Support language

日本語/英語

Japanese/English

アップデートの予定 Updates plan

基本的には「新規機能の追加」等は行わない予定です。

There are no plans to add new features fundamentally.

メンテナンスアップデート期間 Maintenance update schedule

早くとも2023年10月頃までは。

Until around October 2023 at the earliest.

背景 Background

おおまかな経緯　Whole story

2014(?)	アイデアを思い付く。	*Come up with an idea.*
2018	大学学部生として最終学年。	*Final year as an undergraduate student.*
2018-06	このアイデアを卒業研究のテーマにすることを決める。	*Decided to make this idea the theme of my graduation research.*
	研究実験を目的とするソフトウェアとしてiOSアプリの開発を開始。	*Started to develop an iOS application as software for the purpose of research experiments.*
2019-01	主要機能の実装が完了。	*Completed the implementation of the main functions.*
	研究用の実装を省き FlipByBlink ver 1.0 としてAppStoreにリリース。	*Released the application as FlipByBlink ver 1.0 on AppStore.*
2019-03	「随意性瞬目による電子書籍ページめくりUIの提案」という題で卒業研究を学内向けに発表。	*Presented my graduation research titled "Turn a page of ebook by voluntary blinking" to the university.*
	配属された研究室は放任主義だったため、今回の研究/開発における様々な作業は実質的に自分1人だけでほぼ完結していた。	*The laboratory was hands-off policy. As a result, the various tasks of this research and development were done practically by myself.*
2019-05	ver 1.0は非常に使い辛いものだったため、ユーザーインターフェイスを中心に改修し、ver 2.0としてリリース。機能そのものの変更はなかった。	*Because ver 1.0 was very awkward to use, I modified the user interface and released it as ver 2.0. There were no changes to the functions themselves.*
2019 ~ 2021	残念ながらアイデアやアプリが広く認知されることはなかった。	*Unfortunately, the idea and the application did not gain wide recognition.*
2021-12	周知改善を目的としてアプリを大幅に改修してver 3.0としてリリース。	*In order to gain wider recognition, the app was greatly modified and released as ver 3.0.*

both + slow + wink + blink → “boslowink”

2022年現在、「わずかに閉瞼を長くした随意性瞬目」に重点を置いたユーザーインターフェイスや特定の呼び方は(私が知る限り)存在しない。そのため、ver3.0のリリースにあたり「わずかに閉瞼を長くした随意性瞬目」を”boslowink”と命名することにした。

As of 2022, (as far as I know,) there is no such things as UI or specific term that focus on slightly-longish-voluntary-blink. So I provisionally name slightly-longish-voluntary-blink as “boslowink” on the release of ver3.0.

仮説 Hypothesis

boslowinkは下記の特徴を持つような機器でのUIとして有効な手法

- ユーザーへの身体的制約が少ない方が好ましい
- まばたきが検知しやすい
- 対象において必要な機能が単機能
- 対象においてポインティングが必要ない
- 対象が映像かつ遷移時に用いる

boslowink is an effective method of UI for devices that have the below characteristics

- Less physical constraint on the user is preferable
- Easy to detect blinking
- Single function required for the target
- No need for pointing
- Has visual output
- Scene changes when user operates it

具体例 Example

- 電子書籍のページめくりUI
- ビデオゲームやVR等での場面転換UIや位置移動UI
- ハンズフリー操作が求められる場面での確認ダイアログUI
- *UI for turning a page when reading ebook*
- *Scene change UI and position movement UI in video-game and VR, etc*
- *Check dialog UI for situations that require hands-free operation*

このアプリの存在意義　Significance of this app

このアイデアの実現/周知を目的に本アプリは開発/公開されています。

This application has been developed and released with the purpose of realizing and promoting this idea.

boslowinkUIは知的財産？ boslowinkUI is IP?

- このアイデアは私によって考案されたこと。
- 2019年からAppStoreにてアプリを全世界向けに公開をしてきた。既に新規性を失い、さらに一定期間経過したため、私が特許を取得することはほぼ不可能であること。
- アイデアを独占する意志が私には無いこと。
- このアイデアは広く利用される事が好ましいと私は考えていること。

上記の4項目を宣言します。

- *This idea was devised by me.*
- *I have been releasing the app on AppStore for whole world since 2019. It is almost impossible for me to obtain patents because it has already lost its novelty and a certain period of time has passed.*
- *I have no intention to monopolize this idea.*
- *I consider it desirable that this idea be widely utilized.*

I declare above four points.

透明性レポート

Transparency report

AppStoreリリースに関して AppStore release

URL

https://apps.apple.com/us/app/id1444571751

年齢制限指定 App Store age-rating

4+

配信地域 Availability

全て(175)の配信可能な国または地域を対象に配信しています。また、AppStoreで新しく配信可能になる国または地域にも自動的に公開されます。

It is distributed to all (175) available countries or regions. It will also be automatically published to any new countries or regions that become available for distribution on AppStore.

AppleシリコンmacとのAppleシリコンmac互換性 Compatibility with Apple Silicon mac

不可

disable

バージョン履歴 Version history

ver 1.0 : 2019-01-15

最初のリリース

first release

ver 1.0.1 : 2019-02-12

バグ修正

bug fix

ver 1.0.2 : 2019-04-12

広報URLテキストの追記のみ

add URL-text



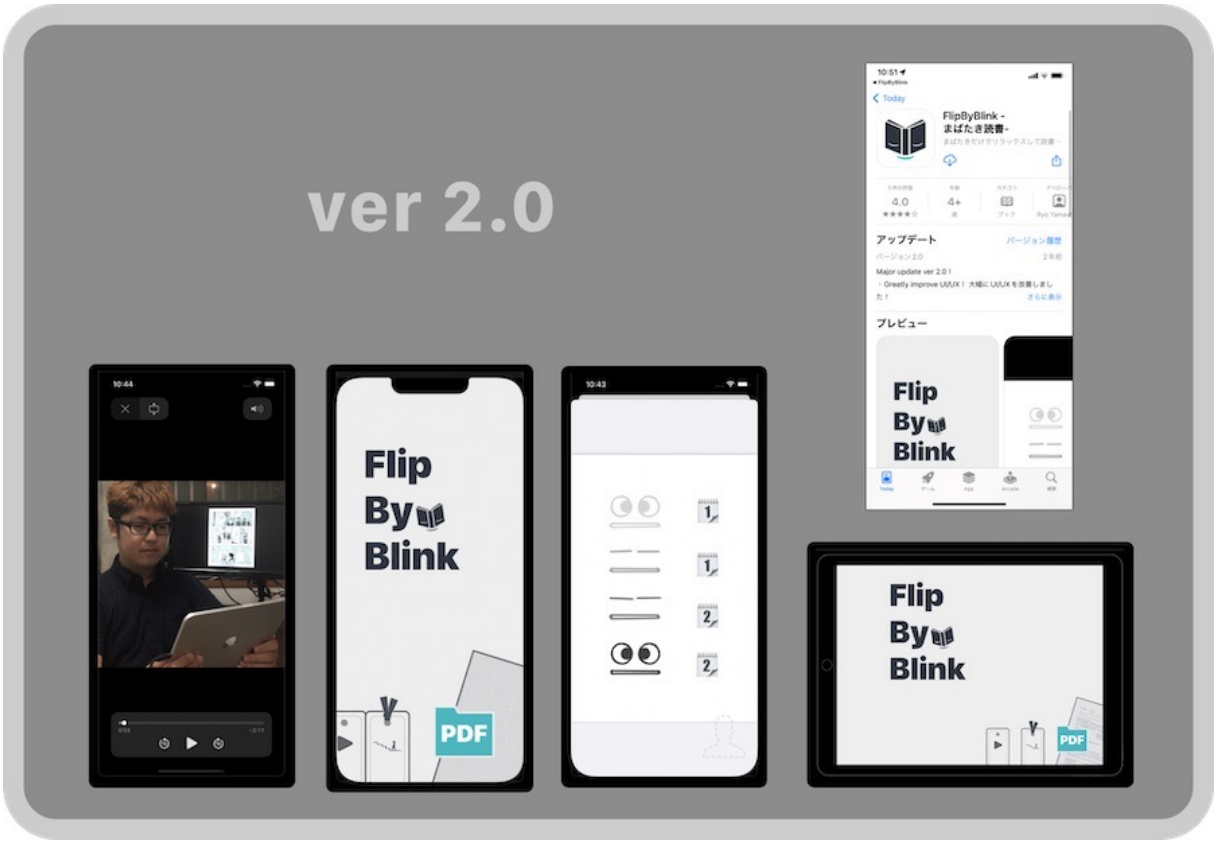
ver 2.0 : 2019-05-23

粗悪なver1.0のUI/UXを大幅に改善

- 操作説明動画を追加
- アイコンのリニューアル
- 機能そのものは変更なし
- サポートOSバージョン iOS 12.1 ~

Improve ver1.0's poor UI/UX design

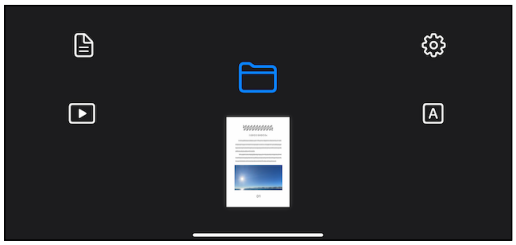
- *Add tutorial video*
- *Remake app icon*
- *There were no changes to the functions themselves*

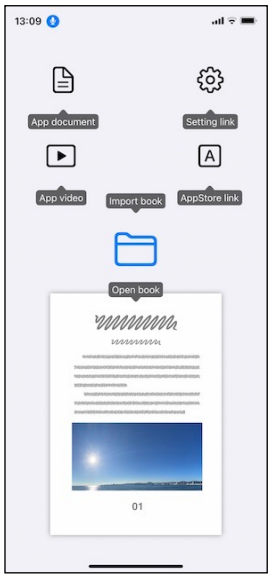


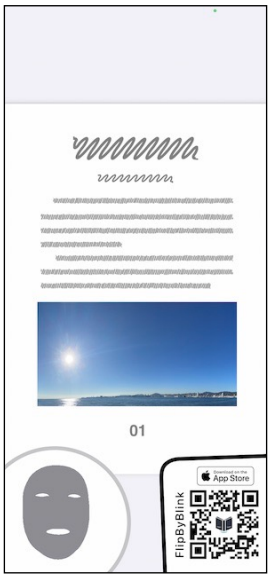
ver 3.0 ：2021-12-06

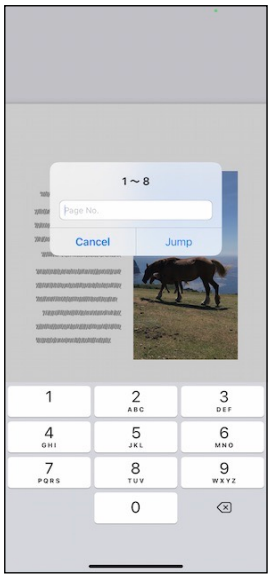
- 最新開発環境への対応
- 新機能
  - プレビュー表示機能
    - 実映像のプレビュー(オプトイン)
    - プレビューを非表示(オプトイン)
  - 最後に読んだページを記録する機能
  - ページジャンプ機能
  - 最後のページで次のページへ進もうとするとアラートを表示
- アプリドキュメントPDFの大幅な追加
- アプリドキュメント動画の大幅な追加
- 英語のサポート
- ダークモードのサポート
- アクセシビリティ互換性向上
- 非対応機種の場合はアラートを表示
- アプリ上にAppStoreリンクを追加
- キーボード入力「→」「←」でもページめくり出来る機能の削除
- 周知を目的とした機能の追加
  - 共有用テロップ表示(オプトイン)
  - プレビュー常時表示(オプトイン)
- 実験的な機能の実装
  - boslowink閾値変更機能(オプトイン)
  - 片目ウインクで1ページ戻る機能(オプトイン)
- サポートOSバージョン: iOS/iPadOS 14.7 ~
- *Adjust latest development environment*
- *New features*
  - *Display preview*
    - *Real preview(opt-in)*
    - *Hide preview(opt-in)*

  - *Save page number*
  - *Jump designate page*
  - *Alert when user flip a next page on last page*
- *Add “App document PDF” greatly*
- *Add “App document video” greatly*
- *Support English*
- *Support dark mode*
- *Improve accessibility compatible*
- *Alert to unsupported device*
- *Add AppStore link in app*
- *Delete turning function by keyboard input “→” / ”←”*
- *In order to gain wider recognition, the app was greatly modified*
  - *Display share-info(opt-in)*
  - *Always display preview(opt-in)*
- *Experimental features*
  - *Setting threshold(opt-in)*
  - *Wink with one eye to return a page(opt-in)*
- *Support OS version: iOS/iPadOS 14.7 ~*









ver 3.0.1 : 2022-01-17

軽微なバグ修正

ファイルをインポートする際に適切にファイルが削除されず端末に残り続ける不具合の修正をしました。

small bug fix

Fixed a bug that files were not deleted properly and remained on the device.

ver 3.0.2 : 2022-03-12

キーボードによるページ移動をサポートしました。

Add keyboard support.

+ 1 page :　→ , d , spacebar , pagedown
+ 5 page :　↓ , s
- 1 page :　← , a , shift*spacebar , pageup
- 5 page :　↑ , w

ver 3.1 : 2022-12-28

ZIPファイルをサポートしました。

サポートOSバージョン: 14.7 → 15.0

Add ZIP file support.

Support OS version: 14.7 → 15.0

ver 3.1.1 : 2022-12-??

軽微なバグ修正。

アプリ名が「FBB Debug」と誤って表示される不具合を修正しました。

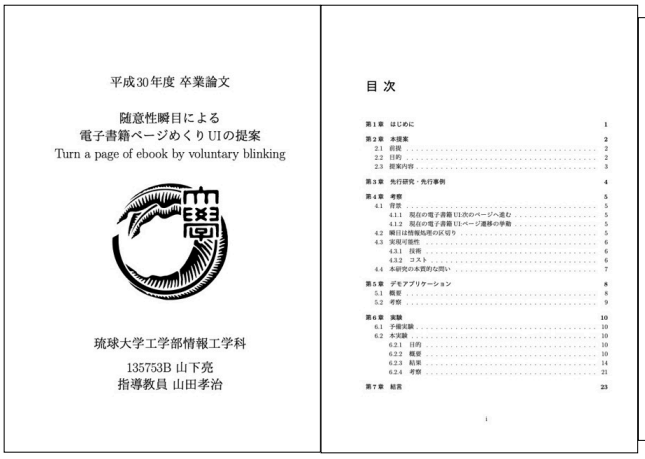
Small bug fix.

Fix a bug that the app name is mistakenly displayed as "FBB Debug".

卒業論文(2019) Graduation thesis(2019)

web上で公開 Public on web

https://github.com/FlipByBlink/Resource_related_to_FlipByBlink



平成30年度 卒業論文

随意性瞬目による
電子書籍ページめくりUIの提案
Turn a page of ebook by voluntary blinking

琉球大学工学部情報工学科

135753B 山下亮
指導教員 山田孝治

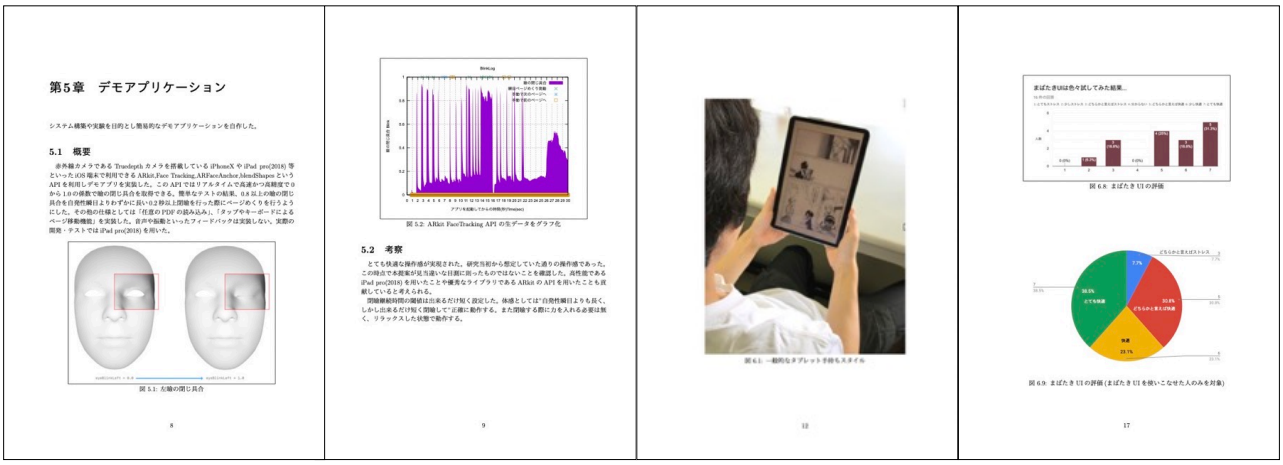
目 次

第1章 はじめに 1
第2章 本提案 2
2.1 前提 2
2.2 目的 2
2.3 提案内容 3
第3章 先行研究・先行事例 4
第4章 考察 5
4.1 背景 5
4.1.1 現在の電子書籍UI次のページへ進む 5
4.1.2 現在の電子書籍UIページ選択の手動 5
4.2 瞬目は情報処理の区切り 5
4.3 実現可能性 6
4.3.1 技術 6
4.3.2 コスト 6
4.4 本研究の本質的な問い 7
第5章 デモアプリケーション 8
5.1 概要 8
5.2 考察 9
第6章 実験 10
6.1 予備実験 10
6.2 本実験 10
6.2.1 目的 10
6.2.2 概要 10
6.2.3 結果 14
6.2.4 考察 21
第7章 結言 23

Abstract

This paper attempts to propose, discuss, and evaluate a prototype of a new UI for e-books which enables users to flip pages with longish voluntary blink. The purpose of this study is to invent a generic UI optimized for turning pages of e-books. This report discusses about proposal. We have created an experimental application for an iPad. The usability of the application has been found to be highly promising after conducting the control experiment between standard UI and new UI proposed. The result of our experiment clearly shows the significance of the proposed method.

本論文では「閉瞼をわずかに継続した随意性瞬目を用いた電子書籍ページめくりUI」の提案・考察・デモアプリ作成・検証を行う。目的は様々なユースケースが存在する電子書籍の特性に沿った汎用的なページ送り方法の発明である。現在の電子書籍UIの課題や、本手法の実現可能性、ユースケース、他分野への応用を考察する。デモとしiPad向けアプリを作成した。そのデモアプリの使い勝手は本提案が想定していた通りのものとなった。またデモアプリを用いて通常の電子書籍UIとの対照実験を行なった。実験結果を通して本提案の有用性を確認した。



第5章 デモアプリケーション

システム構築や実験を目的とし簡易的なデモアプリケーションを自作した。

5.1 概要

赤外線カメラである Truedepth カメラを搭載している iPhoneX や iPad pro(2018) 等といった iOS 端末で利用できる ARKit.Face Tracking.ARFaceAnchor.blendShapes という API を利用しデモアプリを実装した。この API ではリアルタイムで高速かつ高精度でかつ 1.0 の係数で瞼の閉じ具合を取得できる。簡単なテストの結果、0.8 以上の瞼の閉じ具合を自発性瞬目よりわずかに長い 0.2 秒以上閉瞼を行った際にページめくりを行うようにした。その他の仕様としては「任意の PDF の読み込み」「タップやキーボードによるページ移動機能」を実装した。音声や振動といったフィードバックは実装しない。実験の開発・テストでは iPad pro(2018) を用いた。

図 5.1: 左瞼の閉じ具合

8

図 5.2: ARKit FaceTracking API の生データをグラフ化

5.2 考察

とても快適な操作感が実現された。研究当初から想定していた通りの操作感であった。この時点で本提案が見当違いな目指に沿ったものではないことを確認した。高性能である iPad pro(2018) を用いたことや優秀なライブラリである ARkit の API を用いたことも貢献していると考えられる。

閉瞼継続時間の閾値は出来るだけ短く設定した。体感としては「自発性瞬目よりも長く、しかし出来るだけ短く閉瞼して」正確に動作する。また閉瞼する際に力を入れる必要は無く、リラックスした状態で動作する。

9

図 6.1: 一般的なタブレット手持ちスタイル

12

図 6.8: まばたき UI の評価

図 6.9: まばたき UI の評価 (まばたき UI を使いこなせた人のみを対象)

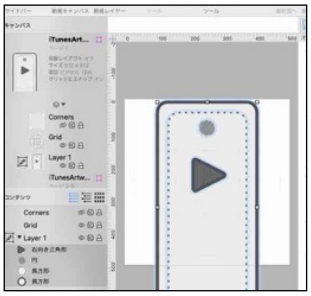
17

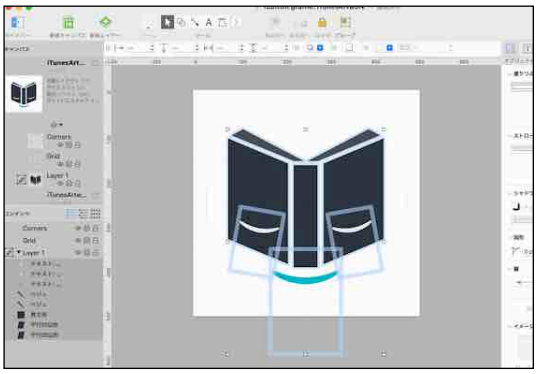
グラフィックデザイン資料 Graphic design resource

















よくありそうな質問

FAQ

障害者向け？ For disability?

ユニバーサルデザインとして障害者の方の利用も想定しています。

As universal design, this app is designed to be used by people with disabilities also.

公式ホームページや問い合わせ窓口は？Official site, contact, etc?

ありません。ただし以下のそれぞれを暫定的にしばらく運用します。

Nothing. But I launch provisionally the following for the time being.

【Twitter】@FlipByBlink

【Mail】pines-dent.0i@icloud.com

【GitHub】https://github.com/FlipByBlink

他のアプリでもboslowinkUIを使いたい Want to use boslowinkUI in other apps

他のアプリ等でboslowinkUIの採用を歓迎します。ただし、AppStoreの規約上の都合により、FlipByBlinkと(直接競合するような)そっくりなアプリをAppStore上で公開する事は控えてください。

参照: https://developer.apple.com/jp/app-store/review/guidelines/#design

I welcome the adoption of boslowinkUI in other apps. However, please refrain from publishing app on AppStore that look exactly like FlipByBlink (or directly compete with it) due to AppStore's terms.

ref. https://developer.apple.com/app-store/review/guidelines/#design

このアプリをAndroidやWindowsではリリースしないのか？ Will you release this app for Android or Windows?

予定は全く無いです。 *Nothing.*

「オープンソースソフトウェア」？ “Open-source software”？

ソースコードを公開していますが、”オープンソースソフトウェア”ではありません。将来的にオープンソースソフトウェアとして運用していく可能性はありますが、現時点では未定です。

The source code is open to public, but it is not “Open-source software". There is a possibility that I will make it “Open-source software" in the future, but it is undecided at this time.

ハンズフリーで1ページ戻る機能も欲しい Want to function for returning a page

アプリのコンセプトから外れるためハンズフリーページ戻り機能はありません。ただ、オプトインの実験的な機能として検討することにしました。ver3.0では片目ウインクで1ページ戻る機能を実装しました。あくまで実験的な機能のため、将来的に仕様の変更や機能の削除があるかもしれません。

There is no feature because it is outside the concept of the app. However, I have tried to consider it as experimental feature(opt-in). Ver3.0 has added a feature to go back a page by winking one eye. Since this is only experimental feature, there may be changes to the specification or removal of the feature in the future.

いつか実装するかもしれないし実装しないかもしれない機能 Features that may or may not be implemented someday

基本的には「新規機能の追加」等は行わない予定です。しかし、ver3.0リリースから一定期間経ったタイミングで、アプリを取り巻く状況が大きく変化した場合はアプリ制作の見直しを行う可能性はあります。その際にもしかしたら実装するかもしれないし実装しないかもしれない機能等を下記に列挙します。

- 詳細なパーソナライズ機能
- 見開き表示
- 音声や振動でフィードバック
- epubのサポート
- macOSのサポート
- 外部モニター対応
- 誤動作抑制機能の実装
- 手を使わず1ページ戻る機能を(正式に)実装

There are no plans to add new features fundamentally. However, there is a possibility that I will revise the app production if the situation surrounding the app changes drastically after a certain period of time from ver3.0 release. The following is a list of new features that I may or may not implement at that time.

- *Personalize feature*
- *Support two pages side-by-side*
- *Feedback sound or vibration*
- *Support epub format*
- *Support macOS*
- *Support external monitor*
- *Feature to prevent malfunction*
- *Officially release function return 1 page without hand*

謝辞 Acknowledgments

ZIP Foundation

version 0.9.16

https://github.com/weichsel/ZIPFoundation

> ZIP Foundation is a library to create, read and modify ZIP archive files.
It is written in Swift and based on Apple's libcompression for high performance and energy efficiency.

MIT License

Copyright (c) 2017-2021 Thomas Zoechling (https://www.peakstep.com)


```
MIT License

Copyright (c) 2017-2021 Thomas Zoechling (https://www.peakstep.com)

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

ソースコード

Sourcecode

GitHub上で公開 Public on GitHub

https://github.com/FlipByBlink/FlipByBlink_ver3

Development environment

- macOS 13.1
- Xcode 14.2
- MacBook Air (M1, 2020)

Project setting

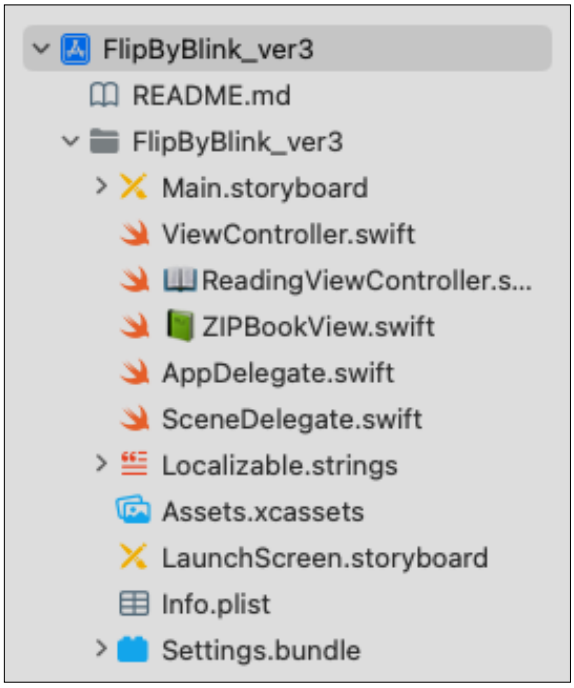
- iOS Deployment Target : 15.0
- Support device : iPhone, iPad
- Localization : Base, English, Japanese

Third party library

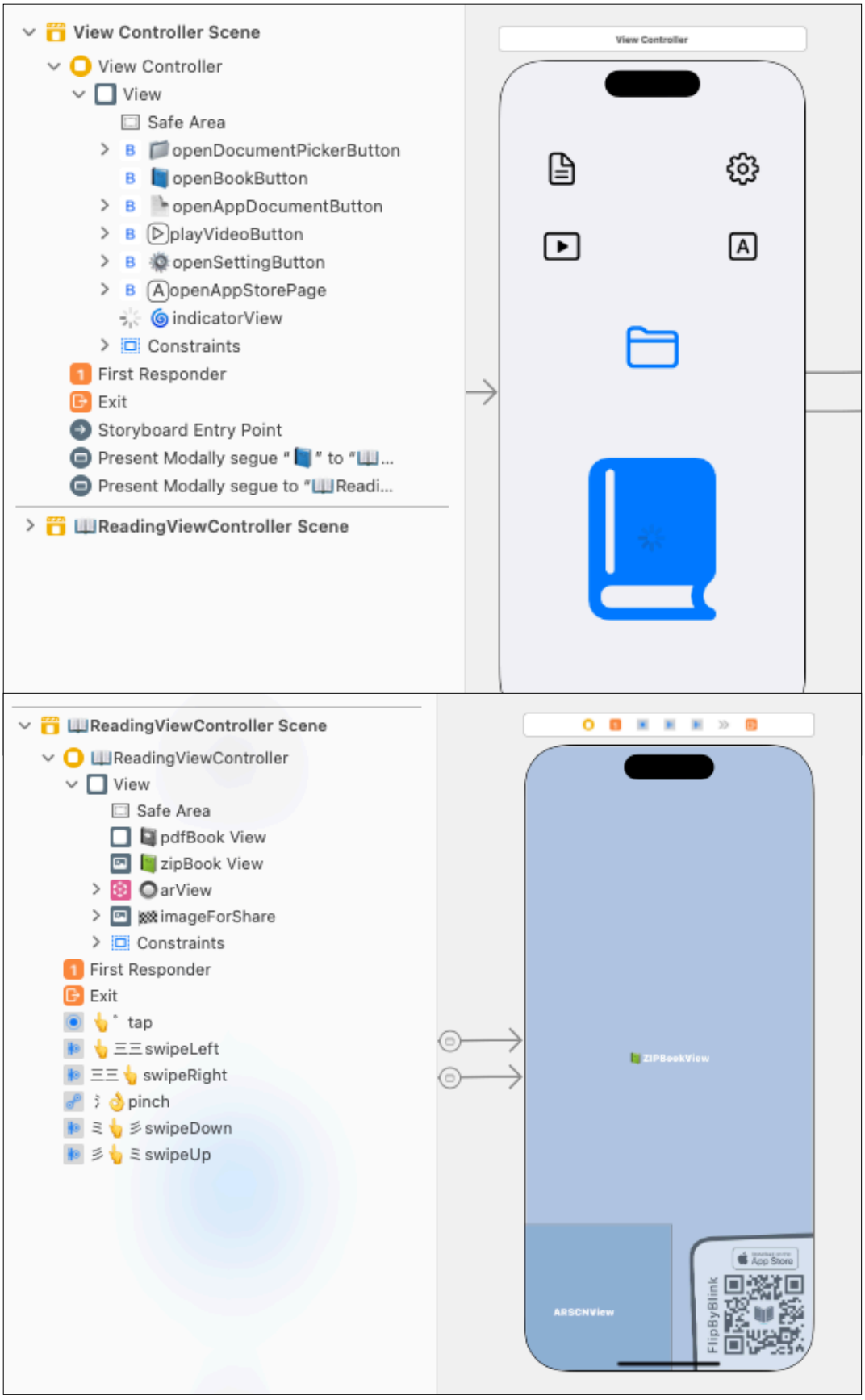
- ZIPFoundation

github.com/weichsel/ZIPFoundation

Repository



StoryBoard


View Controller Scene
View Controller
View
Safe Area
openDocumentPickerButton
openBookButton
openAppDocumentButton
playVideoButton
openSettingButton
openAppStorePage
indicatorView
Constraints
First Responder
Exit
Storyboard Entry Point
Present Modally segue " " to "...
Present Modally segue to "Readi...
ReadingViewController Scene
View Controller
ReadingViewController Scene
ReadingViewController
View
Safe Area
pdfBook View
zipBook View
arView
imageForShare
Constraints
First Responder
Exit
tap
swipeLeft
swipeRight
pinch
swipeDown
swipeUp
ZIPBookView
ARSCNView
App Store
FlipByBlink

ViewController.swift

```
import UIKit
import AVKit
import PDFKit
import ARKit

class ViewController: UIViewController, UIDocumentPickerDelegate {

    private var loadedFile: 🅵ile.MainContent = .presetPDF

    override func viewDidLoad() {
        super.viewDidLoad()
        self.loadFileStatus()
    }

    private func loadFileStatus() {
        self.loadedFile.reload()
        let ⓘmage = self.loadedFile.getCoverImage(of: self.📘
openBookButton.frame.size)
        self.📘openBookButton.setImage(ⓘmage, for: .normal)
    }

    @IBOutlet weak var 📘openBookButton: UIButton! {
        didSet {
            self.📘openBookButton.imageView?.contentMode = .scaleAspectFit
            self.📘openBookButton.layer.shadowColor = UIColor.black.cgColor
            self.📘openBookButton.layer.shadowOpacity = 0.25
            self.📘openBookButton.layer.shadowRadius = 3
            self.📘openBookButton.layer.shadowOffset = .zero
        }
    }

    @IBAction func ▷playVideo() {
        let ⓤrl = Bundle.main.url(forResource: "▶️", withExtension: "mp4")!
        let ⓥc = AVPlayerViewController()
        ⓥc.player = AVPlayer(url: ⓤrl)
        self.present(ⓥc, animated: true)
    }

    // ⚙️
    @IBAction func 🛠openSetting() {
        let ⓤrl = URL(string: UIApplication.openSettingsURLString)!
        UIApplication.shared.open(ⓤrl)
    }

    @IBAction func Ⓐ️openAppStorePage() {
        let ⓤrl = URL(string: "https://apps.apple.com/jp/app/id1444571751")!
        UIApplication.shared.open(ⓤrl)
    }

    @IBOutlet weak var 🌀indicatorView: UIActivityIndicatorView!
```

```
    @IBAction func 📁openDocumentPicker() {
        let ⓣypes = [UTType.pdf, UTType.zip, UTType(filenameExtension: "cbz")!]
        let ⓥc = UIDocumentPickerViewController(forOpeningContentTypes: ⓣypes, asCopy:
true)
        ⓥc.delegate = self
        self.present(ⓥc, animated: true)
        DispatchQueue.main.asyncAfter(deadline: .now() + 1) {
            self.🌀indicatorView.startAnimating()
        }
    }

    // UIDocumentPickerDelegate
    func documentPicker(_ controller: UIDocumentPickerViewController,
didPickDocumentsAt urls: [URL]) {
        do {
            try 🅵ile.store(from: urls.first!)
            self.loadFileStatus()
        } catch {
            print("🚨", #function, error.localizedDescription)
        }
        self.🌀indicatorView.stopAnimating()
    }

    func documentPickerWasCancelled(_ controller: UIDocumentPickerViewController) {
        self.🌀indicatorView.stopAnimating()
    }

    // SceneDelegate.scene(_ :, openURLContexts:)
    func importFileFromOtherApp(url ⓤrl: URL) throws {
        try 🅵ile.store(from: ⓤrl)
        self.loadFileStatus()
    }

    // 📘openBookButton or 📄openAppDocumentButton
    override func prepare(for segue: UIStoryboardSegue, sender: Any?) {
        let 📖vc = segue.destination as! 📖ReadingViewController
        if segue.identifier == "📘" {
            📖vc.pdfDocument = PDFDocument()
            switch self.loadedFile {
                case .presetPDF:
                    📖vc.presentedFile = .presetPDF
                    📖vc.pdfDocument = PDFDocument(data: 🅵ile.presetPDFData)!
                case .importedPDF:
                    if let ⓓocument = PDFDocument(url: 🅵ile.importedPDFURL) {
                        📖vc.presentedFile = .importedPDF
                        📖vc.pdfDocument = ⓓocument
                    }
                case .importedZIP:
                    📖vc.presentedFile = .importedZIP
            }
        } else {
            📖vc.pdfDocument = PDFDocument(url: 🅵ile.appdocumentPDFURL)!
```

```
        }
    }
}

struct 🅵ile {
    enum MainContent {
        case presetPDF
        case importedPDF
        case importedZIP

        func getCoverImage(of ⓢize: CGSize) -> UIImage? {
            let ⓘmageSize = ⓢize.applying(CGAffineTransform(scaleX: 3, y: 3))
            switch self {
                case .presetPDF:
                    let ⓓocument = PDFDocument(data: NSDataAsset(name:
"🅿reset")!.data)
                    return ⓓocument?.page(at: 0)?.thumbnail(of: ⓘmageSize,
for: .mediaBox)
                case .importedPDF:
                    let ⓓocument = PDFDocument(url: 🅵ile.importedPDFURL)
                    return ⓓocument?.page(at: 0)?.thumbnail(of: ⓘmageSize,
for: .mediaBox)
                case .importedZIP:
                    return try! 💾ZIPContents.getCoverImage(of: ⓘmageSize)
            }
        }

        mutating func reload() {
            let 🚩importedPDFExists = FileManager.default.fileExists(atPath:
🅵ile.importedPDFURL.path)
            let 🚩importedZIPExists = 💾ZIPContents.dataExists
            switch (🚩importedPDFExists, 🚩importedZIPExists) {
                case (false, false):
                    self = .presetPDF
                case (true, false):
                    self = .importedPDF
                case (false, true):
                    self = .importedZIP
                default:
                    assertionFailure()
            }
        }
    }

    enum PresentedFile {
        case presetPDF
        case appDocumentPDF
        case importedPDF
        case importedZIP
    }

    static var presetPDFData: Data {
        NSDataAsset(name: "🅿reset")!.data
    }

    static var importedPDFURL: URL {
```

```
        let ⓤrl = FileManager.default.urls(for: .documentDirectory,
in: .userDomainMask)[0]
        return ⓤrl.appendingPathComponent("🅸mported.pdf")
    }

    static var appdocumentPDFURL: URL {
        Bundle.main.url(forResource: "📄", withExtension: "pdf")!
    }

    static func store(from 📦url: URL) throws {
        switch 📦url.pathExtension {
            case "pdf":
                let ⓕm = FileManager()
                if ⓕm.fileExists(atPath: Self.importedPDFURL.path) {
                    try ⓕm.removeItem(at: Self.importedPDFURL)
                }
                try ⓕm.copyItem(at: 📦url, to: Self.importedPDFURL)
                try ⓕm.removeItem(at: 📦url)
                💾ZIPContents.removeUnzipFolder()
                UserDefaults.standard.set(0, forKey: "🧹")
            case "zip", "cbz":
                try 💾ZIPContents.unzipAndSaveFiles(from: 📦url)
                try FileManager.default.removeItem(at: 📦url)
                if FileManager.default.fileExists(atPath: Self.importedPDFURL.path) {
                    try FileManager.default.removeItem(at: Self.importedPDFURL)
                }
                UserDefaults.standard.set(0, forKey: "🧹")
            default:
                assertionFailure()
        }
    }
}
```

📖ReadingViewController.swift

```
import UIKit
import PDFKit
import ARKit

class 📖ReadingViewController: UIViewController, ARSCNViewDelegate, ARSessionDelegate {

    var pdfDocument: PDFDocument = PDFDocument()

    var presentedFile: 🇫ile.PresentedFile = .presetPDF

    @IBOutlet weak var 📗zipBookView: 📗ZIPBookView!

    @IBOutlet weak var 📓pdfBookView: PDFView! {
        didSet {
            self.📓pdfBookView.autoScales = true
            self.📓pdfBookView.displayMode = .singlePage
            self.📓pdfBookView.displaysPageBreaks = false
            self.📓pdfBookView.isUserInteractionEnabled = false
            self.📓pdfBookView.accessibilityElementsHidden = true
            self.📓pdfBookView.document = self.pdfDocument
        }
    }

    @IBOutlet weak var 🔘arView: ARSCNView! {
        didSet {
            self.🔘arView.delegate = self
            self.🔘arView.session.delegate = self
            self.🔘arView.session.run(ARFaceTrackingConfiguration())

            self.🔘arView.layer.cornerRadius = self.🔘arView.frame.height/2
            self.🔘arView.layer.borderWidth = 6
            self.🔘arView.layer.borderColor = UIColor.separator.cgColor

            if UserDefaults.standard.bool(forKey: "👤👷 Real Preview") == false {
                self.🔘arView.scene.background.contents = UIColor.systemBackground
            }

            if UserDefaults.standard.bool(forKey: "👤🚫 Hide Preview") {
                self.🔘arView.isHidden = true
            }
        }
    }

    func renderer(_ renderer: SCNSceneRenderer, nodeFor anchor: ARAnchor) -> SCNNode? {
        if UserDefaults.standard.bool(forKey: "👤🔁 Always Preview") == false {
            DispatchQueue.main.async {
                UIView.animate(withDuration: 2, delay: 2) {
                    self.🔘arView.alpha = 0
                }
            }
```

```
        }

        let 👤 = ARSCNFaceGeometry(device: renderer.device!)!
        👤.firstMaterial?.diffuse.contents = UIColor.systemGray

        if UserDefaults.standard.bool(forKey: "👤👨‍🔧 Real Preview") {
            👤.firstMaterial?.diffuse.contents = UIColor.clear
        }

        return SCNNode(geometry: 👤)
    }

    private var 🕰😑start = Date.now

    private var 🗡😑second: Double {
        if let 🗡 = UserDefaults.standard.string(forKey: "🗡😑second") {
            return Double(🗡)!
        } else {
            return 0.15
        }
    }

    private var 🫀👀value = 0.0
    private let 🗡👀threshold = 0.8

    private var 💤suspend = false

    private var 🕰😉start = Date.now

    private var 🫀😉oneEyeValue = 0.0

    func renderer(_ renderer: SCNSceneRenderer, didUpdate node: SCNNode, for anchor:
ARAnchor) {
        guard let ⓐnchor = anchor as? ARFaceAnchor else { return }
        guard let 👤 = node.geometry as? ARSCNFaceGeometry else { return }
        👤.update(from: ⓐnchor.geometry)


        guard let 🫀👀valueL = ⓐnchor.blendShapes[.eyeBlinkLeft]?.doubleValue else
{ return }
        guard let 🫀👀valueR = ⓐnchor.blendShapes[.eyeBlinkRight]?.doubleValue else
{ return }
        let 🫀👀newValue = ( 🫀👀valueL + 🫀👀valueR ) / 2

        if 🫀👀value < 🗡👀threshold {
            if 🫀👀newValue > 🗡👀threshold {
                🕰😑start = Date.now
            }
        }

        🫀👀value = 🫀👀newValue
```

```
        if 💤suspend { return }

        if 🪒👀newValue > 🗡👀threshold {
            if Date.now.timeIntervalSince(🕰😑start) > 🗡😑second {
                DispatchQueue.main.async {
                    self.goToNextPageWithLastPageAlert()
                }

                💤suspend = true
                DispatchQueue.main.asyncAfter(deadline: .now() + 1 ) {
                    self.💤suspend = false
                }
            }
        }

        if UserDefaults.standard.bool(forKey: "😉 return") {
            let 🪒😉newOneEyeValue = abs( 🪒👀valueL - 🪒👀valueR )

            let 🗡😉oneEyeThreshold = 0.5

            if 🪒😉oneEyeValue < 🗡😉oneEyeThreshold {
                if 🪒😉newOneEyeValue > 🗡😉oneEyeThreshold {
                    🕰😉start = Date.now
                    🕰😉start = Date.now
                }
            }

            🪒😉oneEyeValue = 🪒😉newOneEyeValue

            if 💤suspend == false {
                if 🪒😉newOneEyeValue > 🗡😉oneEyeThreshold {
                    if Date.now.timeIntervalSince(🕰😉start) > 0.5 {
                        DispatchQueue.main.async {
                            self.goToPreviousPage()
                        }

                        💤suspend = true
                        DispatchQueue.main.asyncAfter(deadline: .now() + 1 ) {
                            self.💤suspend = false
                        }
                    }
                }
            }
        }
    }

    @IBAction func 👆˝ tap(_ sender: UITapGestureRecognizer) {
        if sender.location(in: view).x > view.center.x {
            self.goToNextPageWithLastPageAlert()
        } else {
            self.goToPreviousPage()
        }
    }
    @IBAction func 👆三三swipeLeft(_ sender: Any) {
```

```
        self.goToNextPageWithLastPageAlert()
    }

    @IBAction func 三三👆swipeRight(_ sender: Any) {
        self.goToPreviousPage()
    }

    @IBAction func 〉👌pinch(_ sender: UIPinchGestureRecognizer) {
        if sender.velocity > 0 {
            self.goToNextPage()
        } else {
            self.goToPreviousPage()
        }
    }

    @IBAction func 彡👆彡swipeDown(_ sender: Any) {
        self.dismiss(animated: true)
    }

    @IBAction func 彡👆彡swipeUp(_ sender: Any) {
        let 📣 = UIAlertController(title: "1 ～ " + self.pageCount.description,
                                   message: nil,
                                   preferredStyle: .alert)

        📣.addTextField { 🔲 in
            🔲.keyboardType = .numberPad
            let 🧹 = self.currentPageNumber.description
            🔲.placeholder = NSLocalizedString(🧹, comment: "")
        }

        let 🆗 = NSLocalizedString("Jump", comment: "")
        📣.addAction(UIAlertAction(title: 🆗, style: .default) { _ in
            if let 📝 = Int((📣.textFields?.first?.text)!) {
                self.go(to: 📝 - 1)
            }
        })

        let 🆖 = NSLocalizedString("Cancel", comment: "")
        📣.addAction(UIAlertAction(title: 🆖, style: .cancel))

        self.present(📣, animated: true)
    }

    private var canGoToNextPage: Bool {
        switch self.presentedFile {
            case .presetPDF, .appDocumentPDF, .importedPDF:
                return self.📓pdfBookView.canGoToNextPage
            case .importedZIP:
                return self.📗zipBookView.canGoToNextPage()
        }
    }
    private func goToNextPage() {
        switch self.presentedFile {
```

```
            case .presetPDF, .appDocumentPDF, .importedPDF:
                self.📓pdfBookView.goToNextPage(nil)
            case .importedZIP:
                self.📗zipBookView.goToNextPage()
        }
    }

    private func goToNextPageWithLastPageAlert() {
        if self.canGoToNextPage == false {
            let 💬 = NSLocalizedString("🎉 Finish! 🎉", comment: "")
            let 📣 = UIAlertController(title: 💬, message: nil, preferredStyle: .alert)
            present(📣, animated: true)
            DispatchQueue.main.asyncAfter(deadline: .now() + 1.5 ) {
                📣.dismiss(animated: true)
            }
        } else {
            self.goToNextPage()
        }
    }

    private func goToPreviousPage() {
        switch self.presentedFile {
            case .presetPDF, .appDocumentPDF, .importedPDF:
                self.📓pdfBookView.goToPreviousPage(nil)
            case .importedZIP:
                self.📗zipBookView.goToPreviousPage()
        }
    }

    private var pageCount: Int {
        switch self.presentedFile {
            case .presetPDF, .appDocumentPDF, .importedPDF:
                return self.pdfDocument.pageCount
            case .importedZIP:
                return self.📗zipBookView.pageCount
        }
    }

    private var currentPageNumber: Int {
        switch self.presentedFile {
            case .presetPDF, .appDocumentPDF, .importedPDF:
                return self.📓pdfBookView.currentPage!.pageRef!.pageNumber
            case .importedZIP:
                return self.📗zipBookView.representedCurrentPageNumber
        }
    }

    private func go(to ⓟageNumber: Int) {
        switch self.presentedFile {
            case .presetPDF, .appDocumentPDF, .importedPDF:
                if let ⓟage = self.pdfDocument.page(at: ⓟageNumber) {

                    self.📓pdfBookView.go(to: ⓟage)

                }
            case .importedZIP:
                self.📗zipBookView.go(to: ⓟageNumber)
        }
```

```
}

override var prefersHomeIndicatorAutoHidden: Bool { true }

override var prefersStatusBarHidden: Bool { true }

override func viewDidLoad() {
    super.viewDidLoad()
    switch self.presentedFile {
        case .presetPDF, .appDocumentPDF:
            self.📓pdfBookView.isHidden = false
            self.📗zipBookView.isHidden = true
        case .importedPDF:
            self.📓pdfBookView.isHidden = false
            self.📗zipBookView.isHidden = true
            let 🔌 = UserDefaults.standard.integer(forKey: "🔌")
            if let ⓟage = pdfDocument.page(at: 🔌) {

                self.📓pdfBookView.go(to: ⓟage)
            }
        case .importedZIP:
            self.📓pdfBookView.isHidden = true
            self.📗zipBookView.isHidden = false
            let 🔌 = UserDefaults.standard.integer(forKey: "🔌")
            self.📗zipBookView.go(to: 🔌)
            self.📗zipBookView.setLayerShadow()
    }
    UIApplication.shared.isIdleTimerDisabled = true
}

override func viewWillDisappear(_ animated: Bool) {
    super.viewWillDisappear(animated)
    switch self.presentedFile {
        case .importedPDF, .importedZIP:
            let 🔌 = self.currentPageNumber - 1
            UserDefaults.standard.set(🔌, forKey: "🔌")
        default:
            break
    }
}

override func viewDidAppear(_ animated: Bool) {
    super.viewDidAppear(animated)
    if ARFaceTrackingConfiguration.isSupported == false {
        let 😱 = NSLocalizedString("😱 Sorry 😱", comment: "")
        let 💬 = NSLocalizedString("can't work", comment: "")
        let 📣 = UIAlertController(title: 😱, message: 💬, preferredStyle: .alert)
        let 🆗 = NSLocalizedString("OK", comment: "")
        📣.addAction(UIAlertAction(title: 🆗, style: .default))
        present(📣, animated: true)
    }
}
```

```
    override func viewWillTransition(to size: CGSize, with coordinator:
UIViewControllerTransitionCoordinator) {
        super.viewWillTransition(to: size, with: coordinator)
        DispatchQueue.main.asyncAfter(deadline: .now() + 1 ) {
            self.📓pdfBookView.sizeToFit()
        }
    }

    @IBOutlet weak var 🏁imageForShare: UIImageView! {
        didSet {
            if UserDefaults.standard.bool(forKey: "🏁 Display share-info") {
                self.🏁imageForShare.isHidden = false
            }
        }
    }

    override func pressesBegan(_ presses: Set<UIPress>, with event: UIPressesEvent?) {
        super.pressesBegan(presses, with: event)
        for ⓟress in presses {
            switch ⓟress.key?.keyCode {
                case .keyboardRightArrow: self.goToNextPageWithLastPageAlert()
                case .keyboardDownArrow: ⓢkip5Page()
                case .keyboardLeftArrow: self.goToPreviousPage()
                case .keyboardUpArrow: ⓑack5Page()

                case .keyboardD: self.goToNextPageWithLastPageAlert()
                case .keyboardS: ⓢkip5Page()
                case .keyboardA: self.goToPreviousPage()
                case .keyboardW: ⓑack5Page()

                case .keyboardPageDown: self.goToNextPageWithLastPageAlert()
                case .keyboardPageUp: self.goToPreviousPage()

                case .keyboardSpacebar:
                    if event?.modifierFlags == .shift {
                        self.goToPreviousPage()
                    } else {
                        self.goToNextPageWithLastPageAlert()
                    }

                default: print(ⓟress.key.debugDescription)
            }
            func ⓢkip5Page() {
                for _ in 1...5 {
                    self.goToNextPageWithLastPageAlert()
                }
            }
            func ⓑack5Page() {
                for _ in 1...5 {
                    self.goToPreviousPage()
                }
            }
        }
    }
}
```

📗 ZIPBookView.swift

```
import Foundation
import UIKit
import ZIPFoundation

class 📗ZIPBookView: UIImageView {
    private var pageURLs: [Int: URL] = 💾ZIPContents.getPageURLs()
    private var currentPageNumber: Int = 0
    private var preloadedNextPageImage: UIImage? = nil
    private var preloadedPreviousPageImage: UIImage? = nil

    func canGoToNextPage() -> Bool {
        self.pageURLs[self.currentPageNumber + 1] != nil
    }

    func goToNextPage() {
        if let ⓝextPageImage = self.preloadedNextPageImage {
            self.image = ⓝextPageImage
            self.preloadedNextPageImage = nil
            self.currentPageNumber += 1
            self.preloadImages()
        }
    }

    func goToPreviousPage() {
        if let ⓟreviousPageImage = self.preloadedPreviousPageImage {
            self.image = ⓟreviousPageImage
            self.preloadedPreviousPageImage = nil
            self.currentPageNumber -= 1
            self.preloadImages()
        }
    }

    func go(to ⓟageNumber: Int) {
        if let ⓤrl = self.pageURLs[ⓟageNumber] {
            self.image = UIImage(contentsOfFile: ⓤrl.path)
            self.currentPageNumber = ⓟageNumber
            self.preloadImages()
        }
    }

    var pageCount: Int {
        self.pageURLs.count
    }

    var representedCurrentPageNumber: Int {
        self.currentPageNumber + 1
    }

    func setLayerShadow() {
        self.layer.shadowRadius = 3
        self.layer.shadowOffset = .zero
        self.layer.shadowOpacity = 0.25
        self.layer.shadowColor = UIColor.black.cgColor
    }

    private func preloadImages() {
```

```
        let ⓝextPageNumber = self.currentPageNumber + 1
        let ⓟreviousPageNumber = self.currentPageNumber - 1
        if let ⓤrl = self.pageURLs[ⓝextPageNumber] {
            let ⓘmage = UIImage(contentsOfFile: ⓤrl.path)
            ⓘmage?.prepareForDisplay { ⓟreparedImage in
                DispatchQueue.main.async {
                    if self.currentPageNumber + 1 == ⓝextPageNumber {
                        self.preloadedNextPageImage = ⓟreparedImage
                    }
                }
            }
        } else {
            self.preloadedNextPageImage = nil
        }
        if let ⓤrl = self.pageURLs[ⓟreviousPageNumber] {
            let ⓘmage = UIImage(contentsOfFile: ⓤrl.path)
            ⓘmage?.prepareForDisplay { ⓟreparedImage in
                DispatchQueue.main.async {
                    if self.currentPageNumber - 1 == ⓟreviousPageNumber {
                        self.preloadedPreviousPageImage = ⓟreparedImage
                    }
                }
            }
        } else {
            self.preloadedPreviousPageImage = nil
        }
    }
}

struct 💾ZIPContents {
    static func getPageURLs() -> [Int: URL] {
        do {
            return try 📄pageURLs
        } catch {
            return [:]
        }
    }

    static func getCoverImage(of ⓢize: CGSize) throws -> UIImage? {
        if let ⓤrl = try 📄pageURLs[0] {
            let ⓘmage = UIImage(contentsOfFile: ⓤrl.path)
            return ⓘmage?.preparingThumbnail(of: ⓢize)
        } else {
            return UIImage(systemName: "exclamationmark.triangle")
        }
    }

    static var dataExists: Bool {
        FileManager.default.fileExists(atPath: 🔗unzipFolderURL.path)
    }

    static func unzipAndSaveFiles(from ⓤrl: URL) throws {
        if FileManager.default.fileExists(atPath: 🔗unzipFolderURL.path) {
```

```
            try 🗑removeUnzipFolder()
        }
        try FileManager.default.unzipItem(at: ⓤrl, to: 🔗unzipFolderURL)//,
preferredEncoding: .utf8)
        try 🗑removeFilesExpectImages()
    }

    static func removeUnzipFolder() {
        try? 🗑removeUnzipFolder()
    }

    //MARK: private code
    private static var 📄pageURLs: [Int: URL] {
        get throws {
            var ⓢubpaths = try FileManager.default.subpathsOfDirectory(atPath:
🔗unzipFolderURL.path)
            try ⓢubpaths.removeAll { try 🚩isDirecrory($0) }
            ⓢubpaths.sort { $0.localizedStandardCompare($1) == .orderedAscending }
            //alternative: ⓢubpaths.sort { $0.compare($1, options: .numeric)
== .orderedAscending }
            let ⓔmptyIndices: [Int: URL] = [:]
            return ⓢubpaths.reduce(into: ⓔmptyIndices) { ⓟartialResult, ⓢubpath in
                if let ⓘndex = ⓢubpaths.firstIndex(of: ⓢubpath) {
                    ⓟartialResult[ⓘndex] =
🔗unzipFolderURL.appendingPathComponent(ⓢubpath)
                }
            }
        }
    }

    private static var 🔗unzipFolderURL: URL {
        FileManager.default.urls(for: .documentDirectory, in: .userDomainMask)
[0].appendingPathComponent("unzip")
    }

    private static func 🗑removeUnzipFolder() throws {
        try FileManager.default.removeItem(at: 🔗unzipFolderURL)
    }

    private static func 🗑removeFilesExpectImages() throws {
        let ⓢubpaths = try FileManager.default.subpathsOfDirectory(atPath:
🔗unzipFolderURL.path)
        for ⓢubpath in ⓢubpaths {
            if try 🚩isDirecrory(ⓢubpath) == false {
                let ⓤrl = 🔗unzipFolderURL.appendingPathComponent(ⓢubpath)
                let ⓓata = try Data(contentsOf: ⓤrl)
                if UIImage(data: ⓓata) == nil {
                    try FileManager.default.removeItem(at: ⓤrl)
```

```
                }
            }
        }
    }

    private static func 🚩isDirecrory(_ ⓢubpath: String) throws -> Bool {
        let ⓤrl = 🔗unzipFolderURL.appendingPathComponent(ⓢubpath)
        let ⓡesourceValues = try ⓤrl.resourceValues(forKeys: [.isDirectoryKey])
        return ⓡesourceValues.isDirectory == true
    }
}
```

SceneDelegate.swift

```
import UIKit
import PDFKit

class SceneDelegate: UIResponder, UIWindowSceneDelegate {

    var window: UIWindow?

    func scene(_ scene: UIScene, openURLContexts URLContexts: Set<UIOpenURLContext>) {
        if let 🎮 = window?.rootViewController as? ViewController {
            do {
                if let ⓤrl = URLContexts.first?.url {

                    try 🎮.importFileFromOtherApp(url: ⓤrl)

                }
            } catch {
                print("🚨", #function, error.localizedDescription)
            }
        }
    }

    //====="Xcode template source code"====
    func scene(_ scene: UIScene, willConnectTo session: UISceneSession, options
connectionOptions: UIScene.ConnectionOptions) {
        guard let _ = (scene as? UIWindowScene) else { return }
    }

    func sceneDidDisconnect(_ scene: UIScene) {
    }

    func sceneDidBecomeActive(_ scene: UIScene) {
    }

    func sceneWillResignActive(_ scene: UIScene) {
    }

    func sceneWillEnterForeground(_ scene: UIScene) {
    }

    func sceneDidEnterBackground(_ scene: UIScene) {
    }
    //=====================================
}
```

最後のページ Last page